

CSCI/MATH 485 — Assignment #4

Customer Churn Prediction with XGBoost

IBM Telco Customer Churn Dataset

GitHub: github.com/adityaanilraut/MATH-CSCI485_Aditya_Raut/tree/main/Assignment4

Part 1: Data Exploration

Dataset Overview

The IBM Telco Customer Churn dataset contains 7,043 rows and 21 columns. Each row represents a customer; columns capture demographics, subscribed services, billing details, and the churn label.

Features

Column	Type	Description
customerID	str	Unique customer identifier (dropped)
gender	str	Male / Female
SeniorCitizen	int	1 if senior citizen, else 0
Partner	str	Has a partner (Yes/No)
Dependents	str	Has dependents (Yes/No)
tenure	int	Months with the company
PhoneService	str	Has phone service (Yes/No)
MultipleLines	str	Has multiple lines
InternetService	str	DSL / Fiber optic / No
OnlineSecurity	str	Has online security add-on
OnlineBackup	str	Has online backup add-on
DeviceProtection	str	Has device protection
TechSupport	str	Has tech support
StreamingTV	str	Streams TV
StreamingMovies	str	Streams movies
Contract	str	Month-to-month / One year / Two year
PaperlessBilling	str	Enrolled in paperless billing
PaymentMethod	str	Payment channel
MonthlyCharges	float	Monthly bill amount
TotalCharges	str->float	Cumulative bill (stored as string)
Churn	str->int	Target: Yes=1, No=0

Missing Values

No null values were present in the raw dataset. However, TotalCharges is stored as a string and contains 11 whitespace-only entries (customers with tenure = 0). After coercing to numeric, these become NaN and are handled by the median imputer.

Target Variable Distribution

Class	Count	Percentage
No Churn (0)	5,174	73.5%
Churn (1)	1,869	26.5%
Total	7,043	100%

Problem Type & Business Context

This is a binary classification problem. The target variable Churn takes two values: 1 (customer left) and 0 (customer stayed).

Churn prediction is critically important in the telecom industry because acquiring a new customer typically costs 5-7x more than retaining an existing one. By identifying at-risk customers in advance, the company can proactively intervene — for example, by offering contract upgrades, discounts, or improved service bundles. Even a modest reduction in monthly churn can produce significant annual revenue impact.

Part 2: Data Preprocessing

Cleaning Steps

- **Drop customerID:** Unique identifier with no predictive signal. Including it would cause the model to memorise IDs rather than learn patterns.
- **Encode target (Churn -> 0/1):** Scikit-learn and XGBoost require numeric targets. 'Yes' was mapped to 1 and 'No' to 0.
- **Coerce TotalCharges to float:** Stored as object dtype. Whitespace entries (tenure=0) are converted to NaN using errors='coerce'.

Feature Identification

Feature Type	Columns
Numeric (4)	SeniorCitizen, tenure, MonthlyCharges, TotalCharges
Categorical (15)	gender, Partner, Dependents, PhoneService, MultipleLines, InternetService, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies, Contract, PaperlessBilling, PaymentMethod

Train / Test Split

An 80/20 stratified split was used (random_state=42). Stratification preserves the 26.5% churn ratio in both sets, preventing optimistic evaluation.

Split	Rows	Churn Rate
Training	5,634	26.5%
Test	1,409	26.5%

Preprocessing Pipelines

A ColumnTransformer applies separate pipelines to numeric and categorical features, ensuring no data leaks from the test set into training:

- **Numeric pipeline:** SimpleImputer(strategy='median') — median is robust to billing data outliers; the 11 TotalCharges NaNs are filled with the median.
- **Categorical pipeline:** SimpleImputer(strategy='most_frequent') then OneHotEncoder(handle_unknown='ignore', sparse_output=False). handle_unknown='ignore' prevents errors on unseen categories at test time.

Key Code Snippet

```
preprocessor = ColumnTransformer(transformers=[
    ('num', numeric_transformer, numeric_features),
    ('cat', categorical_transformer, categorical_features)
])
```

Part 3: Baseline Model — Logistic Regression

A Logistic Regression classifier (max_iter=1000) was trained inside an sklearn Pipeline that first applies the preprocessor. The pipeline design prevents any information from the test set from leaking into training.

Metrics on Test Set

Metric	Score
Accuracy	0.8034
Precision	0.6520
Recall	0.5561
F1-Score	0.6003
ROC-AUC	0.8426

Classification Report

Class	Precision	Recall	F1-Score	Support
No Churn	0.85	0.89	0.87	1,035
Churn	0.65	0.56	0.60	374
Accuracy			0.80	1,409
Macro Avg	0.75	0.72	0.73	1,409
Weighted Avg	0.80	0.80	0.80	1,409

Confusion Matrix

	Predicted: No Churn	Predicted: Churn
Actual: No Churn	924 (TN)	111 (FP)
Actual: Churn	166 (FN)	208 (TP)

Part 5: Primary Evaluation Metric

Chosen Metric: ROC-AUC

ROC-AUC (Area Under the Receiver Operating Characteristic Curve) measures the model's ability to rank churners above non-churners across all possible decision thresholds. It was selected as the primary metric for three reasons:

- **Class imbalance robustness:** A classifier that always predicts 'No Churn' would score ~73.5% accuracy while being completely useless. ROC-AUC is threshold-independent and is not inflated by the majority class.
- **Business flexibility:** In practice, the decision threshold is tuned after modelling. ROC-AUC evaluates the full trade-off curve rather than a single threshold.
- **Probabilistic scoring:** XGBoost produces calibrated probabilities. ROC-AUC directly rewards a model that assigns higher probabilities to actual churners.

Why Not Accuracy?

Accuracy is inflated by the majority class and gives no indication of how well the model identifies the rare but high-value positive class (churners). ROC-AUC focuses specifically on the model's discriminative power between the two classes.

Part 4: XGBoost + Hyperparameter Tuning

An XGBClassifier was wrapped in the same Pipeline structure as the baseline. GridSearchCV with 3-fold cross-validation was used to search over 16 hyperparameter combinations (2x2x2x2), scored by ROC-AUC.

Hyperparameters Tuned

Hyperparameter	Values Tested	Justification
n_estimators	[100, 200]	Controls boosting rounds. More trees reduce bias but risk overfitting.
max_depth	[3, 5]	Controls tree depth. Depth=3 is more regularised; depth=5 captures richer interactions.
learning_rate	[0.05, 0.1]	Scales each tree's contribution. Lower rates generalise better.
subsample	[0.8, 1.0]	Fraction of rows sampled per tree. Values <1.0 reduce overfitting (stochastic boosting).

Best Hyperparameters Found by GridSearchCV

Hyperparameter	Best Value
learning_rate	0.05
max_depth	3
n_estimators	100
subsample	0.8
Best CV ROC-AUC	0.8509

The optimal configuration favours a conservative, well-regularised model: shallow trees (depth=3), a slow learning rate (0.05), and row subsampling (0.8). This avoids overfitting on the moderately-sized training set.

Metrics on Test Set (Tuned XGBoost)

Metric	Score
Accuracy	0.8077
Precision	0.6807
Recall	0.5187
F1-Score	0.5888
ROC-AUC	0.8480

Classification Report

Class	Precision	Recall	F1-Score	Support
No Churn	0.84	0.91	0.87	1,035
Churn	0.68	0.52	0.59	374
Accuracy			0.81	1,409
Macro Avg	0.76	0.72	0.73	1,409
Weighted Avg	0.80	0.81	0.80	1,409

Confusion Matrix

	Predicted: No Churn	Predicted: Churn
Actual: No Churn	944 (TN)	91 (FP)
Actual: Churn	180 (FN)	194 (TP)

Part 6: Feature Importance & Interpretation

Top 10 Most Important Features (Gain-based)

Rank	Feature	Importance
1	Contract_Month-to-month	0.4153
2	InternetService_Fiber optic	0.1073
3	OnlineSecurity_No	0.0662
4	TechSupport_No	0.0582
5	PaymentMethod_Electronic check	0.0363
6	tenure	0.0327
7	PaperlessBilling_No	0.0245
8	StreamingMovies_Yes	0.0242
9	Contract_Two year	0.0225
10	Contract_One year	0.0189

Plain-Language Interpretation

- **Contract_Month-to-month (0.415):** By far the most powerful predictor. Month-to-month customers have no financial lock-in and can cancel at any time. This single feature accounts for over 40% of the model's decision-making.
- **InternetService_Fiber optic (0.107):** Fiber optic customers churn at notably higher rates than DSL customers, likely due to pricing dissatisfaction or competition from other fiber providers.
- **OnlineSecurity_No (0.066):** Customers without the online security add-on are more prone to churn. Bundled value-added services increase switching costs.
- **TechSupport_No (0.058):** Without tech support, customers encounter unresolved service issues that push them toward switching providers.
- **PaymentMethod_Electronic check (0.036):** Electronic check payers churn at higher rates, possibly indicating a less committed, more price-sensitive segment.
- **tenure (0.033):** Shorter-tenure customers are significantly more likely to churn. Long-term customers have built switching inertia and higher loyalty.
- **PaperlessBilling_No (0.025):** Customers not enrolled in paperless billing churn slightly less, possibly reflecting an older, more stable customer demographic.
- **StreamingMovies_Yes (0.024):** Streaming movie subscribers churn more often, possibly treating the service as optional entertainment rather than a core utility.
- **Contract_Two year and Contract_One year (0.023, 0.019):** Long-term contracts strongly reduce churn probability, reinforcing the importance of contract type.

Business Implication

The company should target retention efforts at month-to-month customers on fiber optic plans who lack security or tech support add-ons and pay by electronic check. Incentivising these customers to upgrade to annual contracts and add bundled services could substantially reduce churn.

Part 7: Final Comparison & Reflection

Model Comparison Table

Metric	Logistic Regression	XGBoost (Tuned)	XGBoost Wins?
Accuracy	0.8034	0.8077	Yes
Precision	0.6520	0.6807	Yes
Recall	0.5561	0.5187	No
F1-Score	0.6003	0.5888	No
ROC-AUC	0.8426	0.8480	Yes

Discussion

Which model performed better? The results are nuanced. XGBoost outperforms Logistic Regression on Accuracy (+0.43%), Precision (+2.87%), and most importantly on ROC-AUC (+0.54%) — our chosen primary metric. However, Logistic Regression edges ahead on Recall (+3.74%) and F1-Score (+1.15%). Since ROC-AUC is threshold-independent and the better holistic measure for this imbalanced classification task, XGBoost is the preferred model overall.

Why might XGBoost perform better on ROC-AUC? This dataset contains non-linear relationships — for example, the compounding interaction between contract type, tenure, and churn risk — that Logistic Regression cannot capture without explicit feature engineering. XGBoost builds an ensemble of decision trees sequentially, each correcting errors of the previous one, allowing it to:

- Automatically capture complex, non-linear feature interactions.
- Handle the threshold-based structure of features like tenure and MonthlyCharges.
- Benefit from hyperparameter tuning to optimise the bias-variance trade-off.

Why does Logistic Regression achieve higher Recall? Logistic Regression tends to spread probabilities more evenly, leading to more churn predictions and thus higher recall. XGBoost's more conservative model is more precise but misses more churners. In practice, the threshold can be tuned post-modelling for either model to achieve any desired recall level.

One Limitation of XGBoost

XGBoost is significantly less interpretable than Logistic Regression. While feature importances provide a global view, it is difficult to explain to a business stakeholder exactly why a specific customer was predicted to churn. Logistic Regression provides clear coefficient-based explanations. Additionally, XGBoost has a larger hyperparameter space and longer training time, especially with exhaustive grid search.

Implementation Summary

Component	Implementation
Data Loading	pd.read_csv() from local CSV file
Cleaning	Drop customerID; map Churn to 0/1; coerce TotalCharges to float
Missing Values	Median imputation for numeric; mode imputation for categorical
Encoding	OneHotEncoder(handle_unknown='ignore', sparse_output=False)
Train/Test Split	train_test_split(test_size=0.2, stratify=y, random_state=42)
Baseline	LogisticRegression(max_iter=1000) in sklearn Pipeline
XGBoost	XGBClassifier(eval_metric='logloss', random_state=42) in Pipeline
Tuning	GridSearchCV(cv=3, scoring='roc_auc', n_jobs=-1) over 16 combinations
Feature Importance	fitted_xgb.feature_importances_ after pipeline extraction
Primary Metric	ROC-AUC (threshold-independent, robust to class imbalance)

GitHub Repository

The full source code, executed notebook, and this report are available at: github.com/adityaanilraut/MATH-CSCI485_Aditya_Raut/tree/main/Assignment4

Key Code Snippets

GridSearchCV setup:

```
param_grid = {
    'classifier__n_estimators': [100, 200],
    'classifier__max_depth': [3, 5],
    'classifier__learning_rate': [0.05, 0.1],
    'classifier__subsample': [0.8, 1.0],
}
grid_search = GridSearchCV(xgb_pipe, param_grid,
    scoring='roc_auc', cv=3, n_jobs=-1)
```

Feature importance extraction:

```
fitted_preprocessor = best_model.named_steps['preprocessor']
fitted_xgb = best_model.named_steps['classifier']
feature_names = fitted_preprocessor.get_feature_names_out()
importances = fitted_xgb.feature_importances_
```